/ 4 P

(c) State the pseudocode for the QuickSort(a, b) algorithm,
where the parameters a, b are indices in a global array $A[1 \,..\, n]$.
(Your code may call the Partition subroutine from the lecture; its pseudocode is printed below.)

QuickSort(a, b)

Partition(a, b, p)

```
1 swap A[p] ↔ A[b]
2 ℓ ← a
3 for i ← a, ..., (b − 1) do:
4 |   if A[i] < A[b] then:
5 |   |   swap A[ℓ] ↔ A[i]
6 |   |   ℓ ← ℓ + 1
7 swap A[b] ↔ A[ℓ]
8 return ℓ
```

TU Ilmenau, Fakultät für Informatik und Automatisierung
FG Algorithmik
Univ.-Prof. Dr. Christoph Berkholz

TECHNISCHE UNIVERSITÄT ILMENAU

Mock Exam

Algorithms

SS 2026 Friday, June 26th, 2026

Duration: 90 Minuten

Remarks

1. The only items that you are allowed to bring into the examination room are:
   - two ballpoint or fountain pens (only in blue or black),
   - your Thoska and an official ID (Personalausweis, drivers licence, passport),
   - something to drink.
2. Write in your answers directly on the work sheets.
   If you need extra space, there are additional sheets at the end of the exam.
3. Your answers, e.g. bounds in big-O notation, should be meaningful:
   "An array of size n can be sorted in time $O(n^{42})$" is not wrong, but not worth a point.

Your Exam ID:

014-A

Exam Review

Sign here *at the exam review* to confirm that you have reviewed our correction of your solutions:

Signature

Date of the Exam Review

Grading

Task	Points
1	/ 8
2	/ 7
3	/ 10
4	/ 11
5	/ 17
6	/ 10
7	/ 10
8	/ 8
9	/ 9
Total	**/ 90**

Grade

Examiner

Task 6: (Huffman Algorithm) / 10 P

The following table gives the frequencies of the letters A, ..., G in an unknown language.
(The entries have already been sorted according to their frequencies.)

(a) Run the *Huffman Algorithm* on this input by filling in the table!
`pred`$[i]$ denotes the parent ("predecessor") of node i in the resulting coding tree.
For `mark`$[i] = 0$, the node i should be the left child of its parent node; for `mark`$[i] = 1$, the node i should be the right child. / 4 P

a_i	G	F	B	A	C	E	D						
i	1	2	3	4	5	6	7	8	9	10	11	12	13
p_i	0.03	0.04	0.05	0.06	0.10	0.29	0.43						
`pred`													–
`mark`													–

(b) Draw the resulting coding tree T!
(Draw the child with edge label 0 as the left child of its parent.) / 4 P

(c) Compute the cost of the computed Huffman code! / 2 P

$\text{Cost}(T, p) =$

This page is intentionally blank. You may use this extra space for your answers.

Task 8: (Dijkstra's Algorithm) / 8 P

(a) Let $G = (V, E, c)$ be a weighted digraph with $n = |V|$ and $m = |E|$. / 1 P
State the running time of Dijkstra's algorithm (using a binary heap as priority queue):

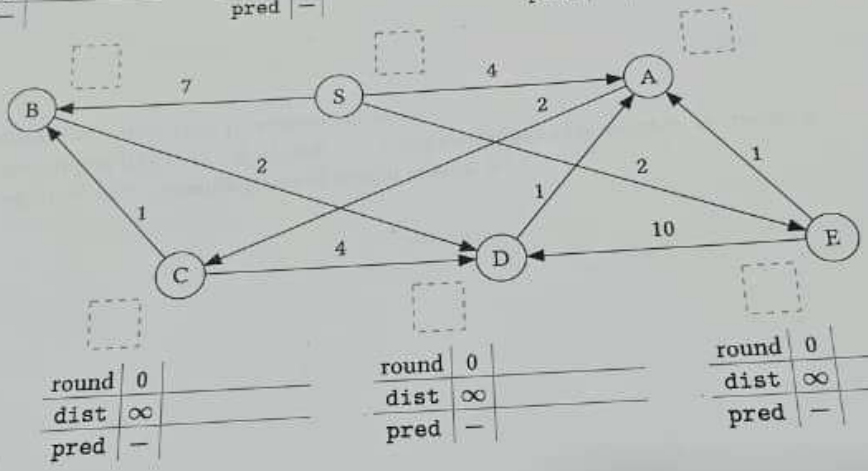
$O($ $)$

(b) Run Dijkstra's algorithm on the weighted digraph below, starting at vertex S. / 5 P

Fill in the fields and tables as follows (see example on the right):
- If node X is processed in round i, write i into the dotted field next to X.
- If node X is found or its distance value `dist`$[X]$ is changed in round i, add a new column to X's table and record
  - the round i,
  - the new distance `dist`$[X]$, and
  - the corresponding (shortest-path tree) predecessor `pred`$[X]$.

(X) 13

round	0	2	5	11
dist	∞	24	12	10
pred	–	F	G	H



round	0
dist	∞
pred	–

round	0
dist	0
pred	–

round	0
dist	∞
pred	–

round	0
dist	∞
pred	–

round	0
dist	∞
pred	–

round	0
dist	∞
pred	–

(c) Suppose we now run Dijkstra's algorithm on a different digraph G starting from a vertex s. / 2 P
At the end of the algorithm, there is a vertex t such that `dist`$[t] = \infty$.
What do we now know about s and t?

(c) Travelling Salesman Problem / 2 P

Input:

Output:

(d) Independent Set Decision Problem / 3 P

Input:

Output:

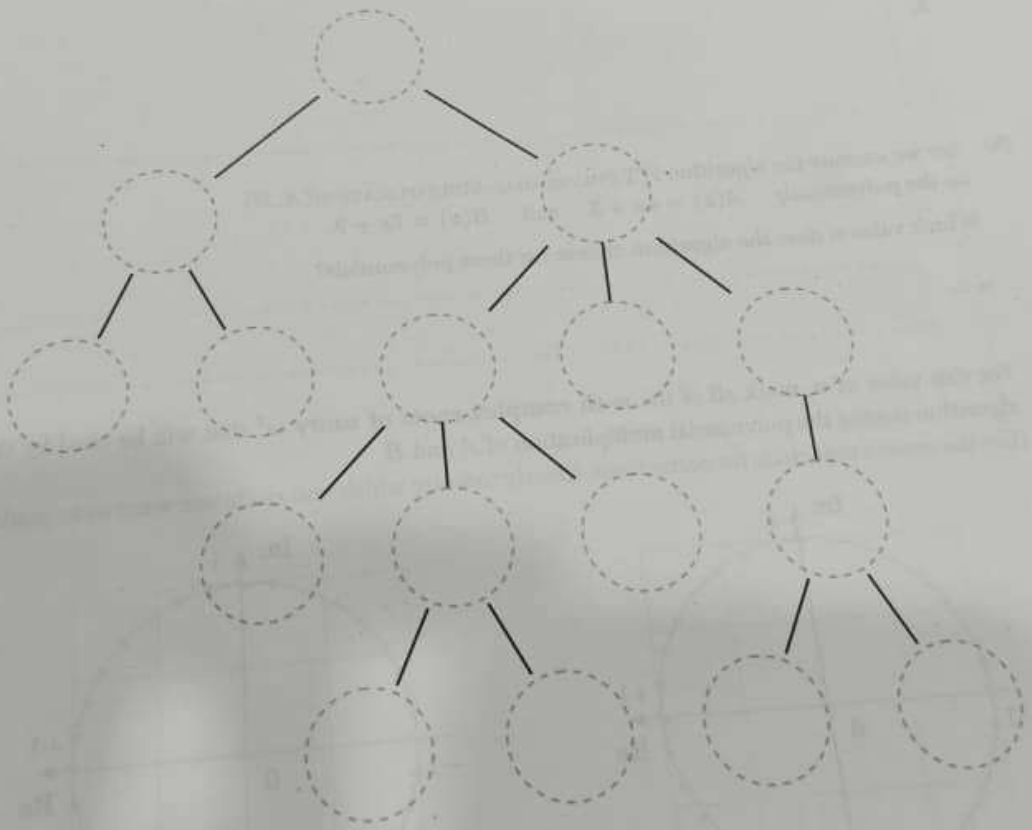
An *independent set* in an undirected graph $G = (V, E)$ is defined as...

(c) Write down the recursive solution that we derived for $\text{MAXIS}(v)$. / 3 P

$$\text{MAXIS}(v) = \begin{cases} \\ \\ \\ \end{cases}$$

Algorithm:

(d) Execute the algorithm that was presented in the lecture on the tree given below by writing the computed values for $\text{MAXIS}(v)$ into the dotted circle at each vertex. / 6 P



This page is intentionally blank. You may use this extra space for your answers.

(c) Show that the product of all the n-th complex roots of unity is $+1$ if n is odd and -1 if n is even. That is, prove that

$$\prod_{k=0}^{n-1} \omega^k = \begin{cases} +1, & \text{if } n \text{ is odd,} \\ -1, & \text{if } n \text{ is even.} \end{cases}$$

Hint: You may use the fact that for any complex number x we have that $x^{ab} = (x^a)^b$ where a and b are integers.

/ 7 P

Proof:

$$\prod_{k=0}^{n-1} \omega^k = \omega^0 \cdot \omega^1 \cdot \omega^2 \cdot \ldots \cdot \omega^{n-1}$$

Task 4: (Maximum Independent Set on Trees)

Let T be a tree with root vertex r.
As in the lecture, we denote the subtree rooted in a vertex v by T_v, and the children of v by $N(v)$.

We want to compute the **size** of a **largest independent set** in T.
In the lecture, we solved the problem with the Dynamic Programming algorithm TreeMaxIS.

Step 1: Formalization + recursive solution:

(a) Describe in words the meaning of the values $\text{MAXIS}(v)$ for a vertex v in the tree.
(How did we define these values in the formalization step?)

/ 1 P

$\text{MAXIS}(v)$ denotes

(b) Suppose we have computed the value $\text{MAXIS}(v)$ for all vertices v.
How can we use this information to find the size of the largest independent set of T?

/ 1 P

(ii) Minimal Spanning Tree **Decision** Problem

- Input: A graph $G = (V, E, c)$ with non-negative integer edge weights, an integer k.
- Output: Is there a spanning tree of weight at most k?

Task 5: (Fast Polynomial Multiplication and FFT)

Let $A(x)$ and $B(x)$ be two polynomials and let $C(x)$ be the polynomial $C = A \cdot B$ of degree $\deg(C) \leq n - 1$, for some power of two $n = 2^d$.
In the lecture you have seen the *fast polynomial multiplication algorithm* FFT-Polynomial-Multiplication(A, B) which takes as input two polynomials $A(x), B(x)$ in coefficient representation, and computes as output the coefficient representation of $C(x)$.

(a) Briefly describe the **three phases** of the algorithm FFT-Polynomial-Multiplication(A, B). For each phase, give the final runtime and name any important subroutines. / 6 P

1.

2.

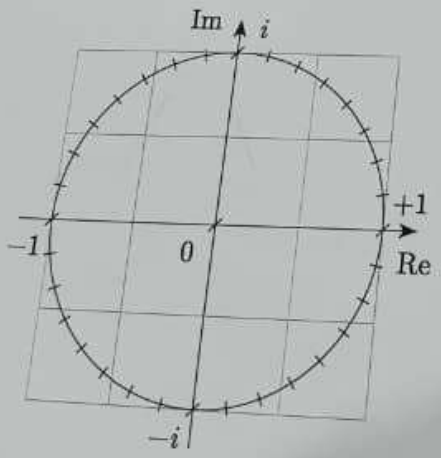
3.

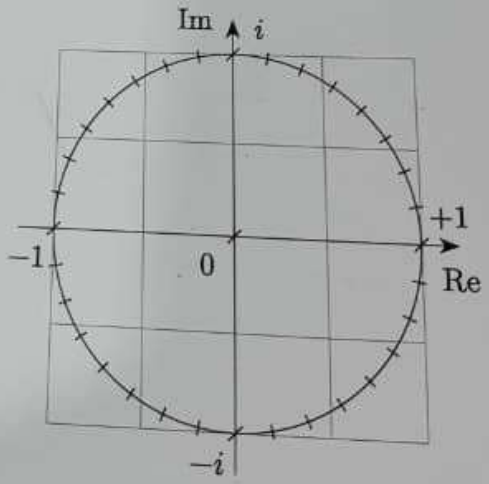
(b) Say we execute the algorithm FFT-Polynomial-Multiplication(A, B) on the polynomials $A(x) = 4x + 3$ and $B(x) = 7x + 9$.

Which value n does the algorithm choose for these polynomials? / 2 P

$n =$

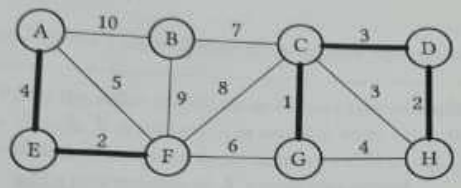
For this value of n, mark all of the **n-th complex roots of unity ω^ℓ** that will be used by the algorithm during the polynomial multiplication of A and B.
(Use the second unit circle for corrections. Clearly indicate which unit circle you want us to grade.) / 2 P





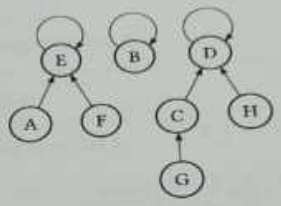
(c) Below is a partial execution of **Kruskal's algorithm** on a graph G: / 4 P

The graph G and the set R.
Edges that have already been added to the set R are marked in **bold**.



$R = \{\{A, E\}, \{E, F\}, \{C, G\}, \{C, D\}, \{D, H\}\}$

The associated **Union-Find data structure** at that time of the exection.



(i) What is the **next edge** e added to R by Kruskal's algorithm?

$e =$

(ii) After e is added to R, the Union-Find data structure has to be updated.
Draw the updated Union-Find data structure!

/ 10 P

Task 7: (Minimum Spanning Trees – Kruskal's Algorithm)

In this task, $G = (V, E, c)$ is an weighted undirected graph with $n = |V|$ many nodes, $m = |E|$ many edges, and edge weight function $c : E \to \mathbb{R}$. You can also assume that G is connected.

/ 2 P

(a) A **spanning tree** of G is defined as an edge set $T \subseteq E$, such that...

What is the definition of the **cost of a spanning tree** T?

$Cost(T) :=$

/ 4 P

(b) Recall that we called a set $R \subseteq E$ *extendible* if there exists an MST T, such that $R \subseteq T$.
Complete the formulation of the **Cut Property**:

"Assume that the following three conditions hold:

1. $R \subseteq E$ is and
2. $(S, V - S)$ is such that , and
3. $e = \{v, w\}$ is an edge with v and w that

Then, ."

This page is intentionally blank. You may use this extra space for your answers.

3.

(b) The MER
In a first
It then re
MERGE sul
Give the re

$T[n] \leq$

This page is intentionally blank. You may use this extra space for your answers.

Task 3: (Problem Descriptions) / 10 P

Complete the following descriptions of various computational problems by filling in the blanks.

(a) EDIT DISTANCE / 3 P

Input:

Output:

The *edit distance* of two strings A, B is defined as...

(b) MATRIX CHAIN MULTIPLICATION / 2 P

Input:

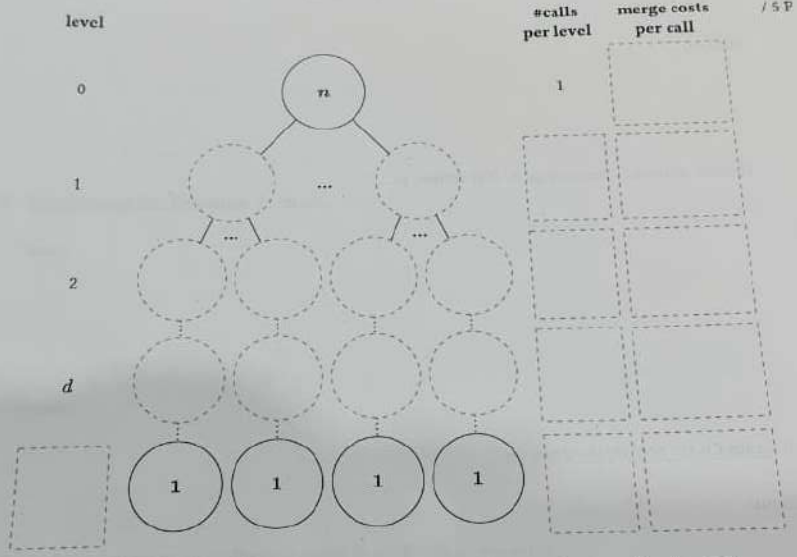
Output:

Task 2: (Recursion Trees) / 7 P

Let $c > 0$ be an integer number. Let T be the following recurrence that describes for all $n = 3^x$ with $x \in \mathbb{N}$ the runtime of a Divide & Conquer-Algorithm $\mathcal{A}$:

$$T[1] = c \qquad \text{and} \qquad T[n] = 5 \cdot T\left[\frac{n}{3}\right] + c \log n.$$

(a) Complete the following recursion tree for T by writing in the **instance sizes**, and for each level the **number of calls** and the **merge costs per call**. Also give the **depth of the tree**, i.e. the level of the tree's leaves. / 5 P



(b) Derive a *non-recursive formula* for the runtime $T[n]$ from the recursion tree, for all $n \geq 2$. (You don't need to simplify the formula.) / 2 P

$T[n] =$

/ 9 P

Task 9: (TSP and NP-hard Problems)

In the lecture, we saw this Dynamic Programming algorithm that solves the Traveling Salesman Problem:

Algorithm (Traveling Salesman Problem)

Input: Graph $G = (V, E)$, cost function $c : E \to \mathbb{R}_0^+$

```
1 Choose start vertex v_1 ∈ V
2 for v ∈ V \ {v_1} do:
3 |  P[∅, w] ← c(v_1, w)
4 for k = 1, ..., n − 2 do:
5 |  for S ⊆ V \ {v_1} and |S| = k do:
6 |  |  for w ∉ S ∪ {v_1} do:
7 |  |  |  P[S, w] ← min_{u∈S} P[S \ {u}, u] + c(u, w)
8 return min_{w∈V\{v_1}} P[V \ {v_1, w}, w] + c(w, v_1)
```

(a) The algorithm stores intermediate values in an array P. What is the **meaning** of the number that is stored at entry $P[S, w]$ (in relation to the start vertex v_1)? / 3 P

/ 8 P

Task 1: (Divide & Conquer)

(a) Name the three phases of a *Divide & Conquer*-Algorithm and briefly describe each. / 3 P

	name	description
1.		
2.		
3.		

(b) *The* MERGESORT *algorithm takes as input an array* $(A[1\,..\,n])$ *and sorts it.*
In a first step, the algorithm splits the array into two arrays $(A[1\,..\,\lfloor \frac{n}{2} \rfloor])$ *and* $(A[\lfloor \frac{n}{2} \rfloor + 1\,..\,n])$.
It then recursively sorts the two arrays before combining them into a single sorted array via the MERGE *subroutine, which runs in time* $O(n)$.
Give the recurrence for the runtime of the algorithm.

$T[n] \leq$ / 1 P

Version 2026-06-24